PFP VI: Retained Failure Evidence and Controlled Reuse

Jeremy H. Carroll

2026-10-02 · Version 0.1.0 · Systems methods preprint · First-publication candidate

Code baseline: `52cb5457f6c485ba548cfd292f6012a941ed22cf`.

Publication package reviewed from `2830d21f695e28dbfcf4f07e18a66da948b4a65b`. The implementation audit remains historical. The default branch's sink has subsequently changed its float encoding and removed the RFC 8785 claim from its encoder docstring. Those later changes are not imported into the baseline argument or its test results. Version 0.1.0 denotes this first public preprint candidate; earlier 1.x labels denote internal drafts.

Abstract

A rejected proposal can explain a defect, reveal a mistaken assumption, or supply a useful negative test. Keeping it also creates new obligations: the record must remain interpretable, its integrity must be checkable, and future reuse must not turn rejection into authority. This paper examines those obligations in the Proposal Fidelity Protocol (PFP), previously called the Symbol Game. The implementation contribution is a source-audited JSONL failure ledger that records structured refusals, chains entry digests, verifies restart state, and emits a count-and-head receipt of its current in-memory state. We describe exactly which modifications its checks detect and why an independently retained anchor is needed when an attacker can rewrite the whole file. A worked two-record example separates local replay, snapshot comparison, and authorized retention. We then specify a controlled reuse procedure using existing detector checks for benign examples, trigger-based diagnostic downgrade thresholds, invalid patterns, and affinity decay. A retained catalog of 61 harvested detector candidates is described only as a data file; no code at the audited commit applies its fixtures. The record store is logically append-only through its API; it does not provide immutable media, authenticated provenance, concurrent-writer safety, or a power-loss durability theorem. No improvement in model accuracy, cost, or discovery yield is inferred from retaining records.

1. Why preserve a failed attempt?

Suppose a candidate code patch edits a protected file, or a proposed mathematical witness refers to the wrong carrier. A host refuses the transition. If the refusal is recorded only as "failed," the next investigator cannot tell which proposal was evaluated, which condition failed, or whether the same failure has already been reproduced.

A useful failure record keeps enough authorized information to reconstruct the attempt. That record can support a regression test and a later comparison of detectors. It is still evidence about an event. It does not establish that the proposal is false under every interpretation, or that a recurring phrase is a reliable detector of future errors.

This paper contributes a concrete storage contract, a retained-anchor integrity argument, and a bounded procedure for promoting a stored example into a proposed reusable rule. The procedure distinguishes three actions: retaining an event, reproducing its failure, and admitting a detector for a declared scope. None of these actions follows automatically from the previous one.

The audited implementation is fixed at commit `52cb5457f6c485ba548cfd292f6012a941ed22cf`. The main sources are `CemeterySink`, its domain record types, and the storage regression suite. The

inherited “cemetery” name means a store for rejected attempts. The concrete data structure is an ordered predecessor chain with a header, not a general graph of arbitrary dependencies. A chain is a special kind of directed acyclic graph when edges are oriented from each new entry to its earlier predecessor; that structural observation supplies neither arbitrary dependency edges nor a semantic mining algorithm.

2. Related work and the threat being addressed

Secure audit logging has long distinguished ordinary local logging from logs designed to retain protections after the logging machine is compromised. Schneier and Kelsey’s construction explicitly addresses compromise and uses cryptographic mechanisms beyond an unkeyed local digest chain. The current PFP store does not implement that construction or inherit its security properties. Schneier and Kelsey, 1999

Certificate Transparency specifies signed tree heads and Merkle inclusion and consistency proofs for a public logging setting. Those mechanisms provide a useful reference for identifying what a local count-and-head record omits: signed producer identity, public witnesses, and a protocol for comparing log views. The PFP store is not a Certificate Transparency log. Laurie, Messeri, and Stradling, 2021

Reflexion uses remembered linguistic feedback to guide later attempts. PFP’s retained failure records can be inputs to a future feedback experiment, but record integrity and improved reasoning are different experimental questions. The present paper studies the former contract and proposes constraints on reuse. Shinn et al., 2023

The immediate threats are accidental mutation, partial truncation, stale records, and inconsistent reconstruction. A stronger adversary may rewrite both the entries and the local header. A still stronger adversary controls the application and any separately retained anchors. Each level needs an explicit interpretation; a checksum alone does not resolve them all.

3. What the store records

`record_refusal` accepts a turn envelope, a refusal reason, optional prompt parameters, and an optional cochain projection. It normalizes supported values, assigns an index and timestamp, and hashes the payload together with the previous entry digest. Fractions are serialized into rational strings in this storage path. The following fields are central:

Field	Meaning and limitation
`entry_index`	Nonnegative position in the ordered chain
`turn_envelope`	Supplied proposal context; not independently authenticated by storage
`refusal_reason`	Nonempty text identifying why the caller recorded the event
`prompt_parameters`	Retained parameters supplied by the caller
`cochain_projection`	Supplied structured values; not a native calculation performed by the sink
`prev_digest`	Commitment to the preceding entry, or the genesis marker
`entry_digest`	SHA-256 of the normalized entry payload
Fixed marker field	Must hold the string `None`; any other value is rejected when an entry is created or reloaded

Each payload also carries a record type and the assigned timestamp. A header records the entry count and current head digest. The exported `TailReceipt` contains those two values, a schema identifier, the same fixed marker, and a SHA-256 digest over those four values. `emit_tail_receipt` reads the sink's cached entries and head; it does not reread the file or recompute the entries. A caller comparing persisted snapshots must first reopen the store or run `verify_integrity` successfully, then emit the observed receipt. This sequence assumes one writer and controlled custody during the check. The sink can check consistency of its entries; it cannot determine whether the caller honestly described an external event.

This distinction matters for mathematical values. A caller can store `{"phi1": "0"}` as text. Its presence and hash do not make it a native functional measurement. A record intended to support such a claim must also identify the actual verifier, input bytes, output bytes, and source/binary bindings, and a separate evaluator must check that relationship.

The store does not enforce a general data-handling policy. The caller must decide which prompts, responses, and parameters it is authorized to retain before calling it. Rejection is not a retention license. The sink's fixed `authority_effect` marker is the literal string `"None"`; it is neither JSON `null` nor producer authentication. Sensitive fields should be excluded, transformed, or handled by an appropriate authorized storage mechanism. A redacted record can support only the evidence claim that its remaining information permits.

4. Logical append and physical persistence

The public API adds entries in increasing order. Physical persistence is different: `_write_file` serializes the header and all retained entries to a temporary file, then replaces the destination with `os.replace`. Consequently "append-only" describes the API's logical history. The file is rewritten on every append.

This design is simple to inspect and supports restart verification, but has clear operational limits. It retains the entries in memory, writes the accumulated contents again, and provides no concurrent-writer lock. Two independent writers can operate from stale snapshots and interfere through the same temporary path. There is no explicit `fsync` call in this path. Process restart tests therefore do not demonstrate persistence through every power failure or storage-device fault.

There is also a failed-write boundary before a restart: `record_refusal` advances the in-memory entry list and head before `_write_file` finishes. If replacement raises an exception, the caller receives no successful append, but the old sink object can emit a receipt for an entry absent from the persisted file. The new package regression injects that one failure and observes this divergence. Discard or successfully reload that object and reconcile the attempted operation before retrying; an exception is not a rollback guarantee. This is a fault-injection example, not a crash campaign.

If each record has a bounded size B and N records are added individually, the accumulated entry bytes written are of order B times $N(N+1)/2$, plus headers, while a single final snapshot occupies order BN bytes. This simple count assumes no batching or compaction and is not a measured benchmark. Bound memory, record size, total retained bytes, and write failures separately.

The encoding helper uses ordinary compact `json.dumps` with sorted keys. Its docstring calls that RFC 8785, but the implementation does not establish the full serialization and Unicode rules required by that specification. This paper calls it the sink's sorted-JSON encoding and binds its source. Interoperability claims require tests against the actual profile and supported value domain. For example, Python's code-point key order differs from JCS's UTF-16 code-unit order for some

supplementary characters, and the baseline encoder leaves Python's permissive non-finite-number option enabled. JCS requires rejection of NaN and infinity. These are concrete differences, not an assumption that sorted keys provide standards conformance. RFC 8785

Normalization is also lossy: `Fraction(1, 3)` and the string `"1/3"` become the same stored value; integer key `1` and string key `"1"` collide after key stringification. The sink therefore retains normalized proposal information, not an injective encoding of every caller value. Preserve separately authorized raw bytes and their type/encoding contract when exact original reconstruction is required.

The returned record is a frozen dataclass, but nested mappings are not thereby made transitively immutable. Consumers should reconstruct evidence from verified persisted bytes and avoid treating an in-memory tuple of records as an immutable storage medium. These limitations do not invalidate a scoped restart check; they define its scope.

5. A retained-anchor integrity argument

Let P_i be the normalized payload of entry i, including its predecessor digest, and let h_i be SHA-256 of that payload's encoded bytes. The header claims a length n and a head h_(n-1). Verification reparses each entry, checks its index and predecessor link, recomputes h_i, and compares the resulting count and head to the header. The exercised empty-file, chain, count, and head checks produce typed integrity or truncation errors. This is not a claim that every malformed Python or JSON value follows that error taxonomy.

Under the assumptions that the encoding is deterministic for admitted values, the parser checks the relevant fields, and a changed payload cannot be made to collide with the expected digest, changing a protected entry payload while keeping its original downstream anchor is detectable. Reordering or removing an interior record also breaks an index, predecessor link, or the expected head. This is a conditional integrity argument, not a formal verification of all parser and filesystem behavior.

The location of the anchor matters. If an attacker removes the last record but leaves the header unchanged, the local count/head comparison detects truncation. If the attacker rewrites the header to match a shorter consistent prefix, that prefix can pass local verification. If the attacker rewrites every entry and recalculates every digest, local self-consistency can also hold.

Retaining a prior `TailReceipt` independently enables a caller to ask whether a reconstructed file has the same ordered normalized-entry count and head. It does not establish raw-file or complete header identity. For example, the sink normalizes a refusal reason through `str`, so the JSON number `42` and string `"42"` can produce the same protected payload; header genesis metadata and unknown header fields are not committed by this comparison. The baseline loader also does not validate the header's `schema` value. Replacing that value can leave the count/head comparison unchanged; an emitted receipt's schema does not retrospectively validate the file header. A caller requiring exact file bytes must retain and check an independent raw-file digest as well. The current sink exposes the receipt, but it does not automatically retrieve an independently trusted copy or perform a signed freshness protocol. The consumer must preserve the anchor and compare count and head. A valid later extension intentionally has a different head; proving ancestry of that extension requires inspecting the chain against the retained prefix. Unequal heads alone show only that the two snapshots differ.

Persisted modification	Baseline reload	Comparison with retained count/head
Change a protected normalized payload, keep old digest	Rejects	No verified new snapshot
Remove suffix, keep old header	Rejects	No verified new snapshot
Remove suffix and write a matching header	Can pass	Different from the retained longer snapshot
Add a valid next entry	Passes	Different; inspect the retained prefix to establish extension
Change unbound header metadata or JSON whitespace	Can pass	Can match; raw-file digest can differ

The hash assumption concerns replacement of protected encoded payloads against a retained digest. It does not turn normalizations that already map different inputs to the same payload into detectable changes, and it does not authenticate the anchor's custodian. A self-digest of a receipt can be recomputed by anyone who can replace that receipt.

The same boundary applies to dates. A timestamp records a value written by the application. The store does not authenticate a clock or prove that the event happened at that time.

6. Worked example: two refusals and three modifications

Use a fresh writable directory and synthetic, nonsensitive data. The following example calls the actual storage API:

```
from pathlib import Path
from omega_engine_symbol_game.infrastructure.persistence import cemetery_sink

CemeterySink = cemetery_sink.CemeterySink

path = Path("fresh-failure-ledger") / "failures.jsonl"
sink = CemeterySink(path)
sink.record_refusal(
    {"task_id": "toy-1", "candidate": "edit README.md"},
    "PROTECTED_PATH",
    prompt_parameters={"transport": "scripted_fixture"},
)
sink.record_refusal(
    {"task_id": "toy-1", "candidate": "use an absent route"},
    "NO_NATIVE_PORT",
    prompt_parameters={"transport": "scripted_fixture"},
)
anchor = sink.emit_tail_receipt()
reopened = CemeterySink(path)
observed = reopened.emit_tail_receipt()
assert (observed.entry_count, observed.head_digest) == (
    anchor.entry_count, anchor.head_digest
)
```

The example compares the normalized-entry snapshot after reopening; it does not run a code ownership checker or native route. The refusal reasons are deliberately constructed labels. A real campaign must bind the host decisions from which those reasons were derived. Timestamp-dependent digests are obtained from execution, not hardcoded in the manuscript.

In the first modification, change `PROTECTED_PATH` in the first persisted entry while leaving its digest unchanged. Reopening rejects the changed payload with `LedgerIntegrityError`. In the second, delete the final entry while retaining the original header. Reopening rejects the count/head discrepancy with `LedgerTruncationError`. The storage tests contain these classes of restart and tamper checks, along with extra-key, malformed UTF-8, invalid-count, and fixed-marker cases.

In the third modification, construct a complete internally consistent shorter file. It reopens without error, so local integrity is no longer enough. A caller comparing its count and head with the independently held two-entry anchor can observe that the file does not match the retained snapshot. The anchor must actually be held outside the rewriting adversary's control; saving it beside the file under the same untrusted custody does not provide that assumption automatically.

This example is small enough for a new reader to inspect every record. It also shows why the useful output is a specific comparison, such as "these entries match this retained snapshot," rather than a blanket claim that the storage is immutable.

7. From a retained event to a reusable detector

A proposed reuse pipeline starts with a query over retained records, groups events by a declared key, and selects candidates for reproduction. Grouping by a digest is a conditional identity test for the particular covered bytes and encoding, under the stated hash assumption. A hash of a label or normalized entry does not identify the entire original proposal or its semantic defect. Grouping by a normalized feature or presentation key establishes only that chosen equivalence relation. Semantic equivalence needs a separate checker.

The next step reproduces the failure using the original input and checker bindings. A failure caused by missing infrastructure must remain a missing infrastructure case. It should not become a detector that declares similar mathematical witnesses invalid. If the original bytes were not retained under policy, record the replay limitation rather than filling them in from memory.

A reproduced defect can become a proposed rule. The rule is tested on the failing case and on valid near misses before any enforcement action is considered. Its provenance includes the source records, predicate, fixture bank, permitted action, and scope. Reuse is a new evaluation event, with a new receipt. The current sources supply individual checks for this process; this paper does not claim an integrated automatic miner with measured discovery yield.

The clonal-selection module illustrates one such check. It compiles a candidate regular expression and compares it with benign self-panel examples. A matching clone produces a kill-condition record naming both clone and parent. An invalid expression returns `INVALID_MATCH_PATTERN`; a Boolean "not killed" value must not be misread as admission when that refusal is present. A clean result means only no match on the supplied panel. The method returns a decision and record; it does not mutate the frozen parent or clone, remove them from a repository, or demonstrate that a host applied the proposed purge. That consumer action is a separate integration obligation. Pattern execution also needs appropriate input-size and resource limits; a syntactically valid regular expression is not a bounded runtime guarantee.

The self-panel regression identifies 40 distinct base contents inside 1,000 stored examples. A publication must report that effective diversity, rather than presenting synthetic repetitions as independent coverage. Adding new domains needs new valid examples and a renewed check.

7.1 The retained detector-candidate catalog

The repository retains a JSON file of harvested detector candidates, `_intelligence/_antibodies/pharmacy_catalog.json`. Its inherited name, pharmacy catalog, is a label for a file of records; the file is not an active filter. At the audited commit it holds 61 entries with 14 distinct `failure_class` values. Its `pharmacy_status` field is `candidate` for 56 entries and `active` for 5.

`failure_class` value	Entries	`pharmacy_status`
`defect:defect:0` to `defect:defect:4` (five classes)	9 each, 45 in total	candidate
`defect:h_boundary`	4	candidate
`general_obstruction`	3	candidate
`architecture:clean_arch_inward_dependency_violation`	3	candidate
`architecture:circular_dependency`	1	candidate
`algebraic_defect:energy_projector_sign_inversion`	1	active
`algebraic_defect:minimizer_bitmask_convention_inversion`	1	active
`resource_exhaustion:reasoning_trace_exceeds_completion_window`	1	active
`syntax_error:unterminated_string_literal`	1	active
`type_error:unsupported_operand_int_and_list`	1	active

Each entry has 15 fields, including `negative_fixture`, `positive_fixture`, and `evidence_sha256`; every entry has all three. The fixtures are short strings. In 49 entries they are placeholder comments: a `# Triggering defect:` line that repeats the defect label, and `# Repaired exact state`. The other 12 entries hold a descriptive phrase or a code fragment of at most four lines. The file binds no fixture to a checker, to input bytes, or to a test that reproduces the failure, and it records no reproduction or near-miss result.

Each `evidence_sha256` value is a distinct 64-digit hexadecimal SHA-256 digest. The file does not record which bytes a digest covers or which code wrote an entry. The two harvesting implementations at the audited commit hash short identifying strings: one joins the defect label with the run and task identifiers, and the other also prefixes the entry identifier. For 19 entries, namely 15 of the `defect:defect:N` entries and the four `defect:h_boundary` entries, the stored value equals the SHA-256 of the defect label joined with a run identifier and task identifier that appear in run records retained at that commit. None of these inputs is a reproduction receipt. As with the head digest in Section 5, such a value supports comparison only against an independently retained anchor. It does not make an entry immutable, and it does not show that a reproduction exists.

No code at the audited commit matches incoming proposals against these fixtures, executes a fixture, or derives a repair from a positive fixture. The nearest code found, a prompt-diagnosis service, tests whether an entry's `target_pattern` occurs as a substring of a prompt; it reads neither fixture. The counts above therefore describe the file. They are not measures of detector coverage or effectiveness. Admitting any entry as a detector would require the reproduction and near-miss steps described above.

8. Monitoring overactive rules and decay

The regulatory supervisor computes a toxicity index from trigger records. With the explicit trigger interface, each supplied record states whether the rule's pattern matched and whether the second signal of the two-signal gate was also present. The index is the number of pattern-only records

divided by the number of supplied records. At the default threshold 1/5, toxicity at or above the threshold synthesizes a record proposing a downgrade to `DIAGNOSTIC_ONLY`. Synthesis alone does not update the rule or host policy. Five records containing one pattern-only case give exactly 1/5 and therefore meet the threshold. One such case among six gives 1/6 and remains below it.

These are policy arithmetic, not an estimated population false-positive rate. The trigger labels must themselves be justified. They describe signal presence, not an independent oracle judgment that a proposal is valid or suppressed. Records with neither signal still enter the explicit-interface denominator. For one pattern-only record, two dual-signal records, and three no-signal records, this interface reports 1/6; the fraction conditional on a pattern match would instead be 1/3. Neither denominator is automatically the valid, eligible-proposal denominator of PFP III's false-suppression experiment. An empty trigger list produces zero by convention, not evidence of safety. A different input interface using an unlabelled prompt corpus has different information available: every match counts as pattern-only, so the score is 0 when nothing matches and 1 otherwise. That score cannot be substituted for an oracle-labeled error rate. The regression suite makes this distinction observable.

Affinity decay provides another bounded mechanism. For idle count t and positive fixed interval tau and a nonnegative initial score, the implementation halves a rule's score once for each newly completed interval: a call that moves the idle count from t to t' applies the factor (1/2)^k with k = floor(t'/tau) − floor(t/tau). Twelve idle runs with tau=5 multiply the starting score by 1/4, whether recorded in one call or batches of 3, 4, and 5. This batching invariance follows from telescoping the differences of completed interval counts and is checked by the tests. It assumes tau remains fixed and no stimulation, reset, or manual score update occurs between batches. Activity expiry is a separate policy: the rule is marked inactive once its idle count reaches its configured `half_life_runs` value.

Neither thresholding nor decay establishes that a rule improves reasoning. They regulate a proposed detector repertoire. Effectiveness and safety require the held-out, independently labeled experiment specified in PFP III. A retained failure can be valuable for diagnosis even when the proposed detector derived from it is rejected.

9. Reproducibility and evidence classification

Distinguish the manuscript's baseline from a current-checkout regression run. To reproduce the historical implementation, use the pinned commit in a separate checkout and record the exact source hashes. Copy the revised paper package to the corresponding deposit path there if its added tests are absent at that historical commit. To test the current branch, report that checkout's own source hashes and any changed contracts instead of labelling the result a baseline replay.

Collect and run the focused suites from the root of a full repository checkout, using Python 3.12 (on Windows, `py -3.12` in place of `python`):

```
python -m pytest omega_engine_symbol_game/tests/test_cemetery_sink.py --collect-only -q
python -m pytest omega_engine_symbol_game/tests/test_cemetery_sink.py -q
python -m pytest omega_engine_symbol_game/tests/test_immune_defect_regressions.py --
    collect-only -q
python -m pytest omega_engine_symbol_game/tests/test_immune_defect_regressions.py -q
```

The first suite tests the production sink; the second targets concrete rule maintenance defects and boundary cases. Use an explicit writable temporary location where required. Record commands, exits, interpreter, source hashes, and exact test outcomes. No provider call or model download is required for these focused checks.

The package test `code/tests/test_006_worked_example.py` executes the Section 6 block as printed, the three modifications described there, and the number-versus-string reason case from Section 5 against the repository sink. It also exercises unbound header metadata, whitespace normalization, cached receipt emission, failed replacement, and mutation of nested mappings. These cases establish the specific evidence boundaries described above. It needs the full repository checkout and skips, stating that reason, when the sink source is absent. It writes ledgers only under pytest's temporary directory. The package test `code/tests/test_006_catalog_file.py` recomputes the Section 7.1 counts, fields, placeholder fixtures, and digest reconstructions from the catalog bytes read through Git at the audited commit. The Git path requires that historical object and a `git` executable; the test falls back to the identical package-local catalog snapshot when Git or that object is unavailable. The snapshot is bound to the baseline path, Git blob, and SHA-256 in `code/fixtures/README.md`; the tests check its digest before using it. This makes the catalog arithmetic rerunnable from the deposit alone, without presenting the catalog as an executable detector bank.

The paper-local `code/tests/test_006_epistemic_immune.py` defines independent toy classes. It is retained as an educational illustration and is not evidence that the production sink or immune pipeline behaves the same way. Its copied positive/zero-defect rule also differs from the actual two-signal gate. It must not supply production integration counts.

Current execution receipts must identify which checks actually ran. Historical receipts and stored outputs remain historical. A successful restart test establishes the exercised process/reconstruction behavior; no crash-consistency, power-failure, or concurrent-writer campaign is claimed. There is also no empirical corpus of mined conjectures presented as a validated result in this paper.

10. Limitations and next experiments

The storage implementation is suitable for bounded, inspectable local runs with a single writer and controlled custody. Its trust boundary is too weak for claims of permanent immutable retention or authenticated multi-party logging. Source inspection also identifies a naming mismatch between the sorted-JSON encoder and RFC 8785; interoperability should be resolved before using a different implementation as a verifier.

The proposed reuse procedure has meaningful unresolved work: retain anchors under an explicit trust policy, bind actual host decisions into entries, demonstrate a deterministic reproduction path for each mined case, and measure rule behavior on independent valid near misses. Any future integrated miner must report both useful discoveries and harmful suppressions under a frozen evaluation plan.

The practical result is narrower and useful: a failed attempt can become an inspectable, source-bound research object. Preserving that object supports reproduction and review while leaving its mathematical meaning and future utility open to independent evaluation.

This paper assigns no authorization and changes no jurisdiction. It carries no operational authority or institutional standing.

Verification for this revision

The package verifier (`verify_deposit.py`) checks the manifest and the two-way file census, and `tools/rebuild_release.py --check` checks the release archive against both manifests. The fresh `docs/first_publication_validation_2026_10_02.json` receipt and accompanying review

identify commands, tool versions, source hashes, and exact exercised scope. Baseline tests were run against byte-verified materialized sources, not a full-repository checkout. Catalog arithmetic uses the bound data snapshot. No current-branch integration or provider trial is inferred. Historical receipts in this package keep their original dates and scope and are not current execution evidence.

References

- Schneier, B., and Kelsey, J. (1999). Secure Audit Logs to Support Computer Forensics. ACM Transactions on Information and System Security 2(2), 159–176. DOI 10.1145/317087.317089.
- Laurie, B., Messeri, E., and Stradling, R. (2021). Certificate Transparency Version 2.0. RFC 9162 (Experimental), Internet Engineering Task Force, December. DOI 10.17487/RFC9162.
- Shinn, N., Cassano, F., Berman, E., Gopinath, A., Narasimhan, K., and Yao, S. (2023). Reflexion: Language Agents with Verbal Reinforcement Learning. arXiv preprint arXiv:2303.11366, version 4.
- Rundgren, A., Jordan, B., and Erdtman, S. (2020). JSON Canonicalization Scheme (JCS). RFC 8785 (Informational), Internet Engineering Task Force, June. DOI 10.17487/RFC8785.

Implementation links throughout the paper bind the storage and detector claims to the stated source baseline. They do not replace independently retained execution or integrity anchors.